

Assignment Breakdown Priorities

<https://chatgpt.com/c/6a9d8f11-7b24-83eb-b8df-e2eee6f6908f>

AI_Data_Engineer_Candidate_Assignment(1).docx here is my assingme...
Yes. The assignment makes the prioritization unusually clear because it...
lets find API that has fewest barriers im thinking sports data
Sports is a good direction here because it can give you a clear business...
ive looked throught the website and i prefer to analyze the premier leau...
Thinking 🌐 Searching the web...
match_data_pl_2.json here is a partial json response from this api for a...
This partial response is actually enough to confirm that the dataset is...
lets desing a simple extarction module (extract.py) that pulls the data...
I'd keep extract.py responsible for only three things: discover current...
for a given matchweek - there may be games yet to be played (differen...
Exactly. In that case, I'd separate matchweek completeness from which...
generate the extract module
I'll generate it with the responsibility split we agreed on: extract.py will...
this module is heavy and abit over complicate - use helper function to g...
Agreed. We can simplify it a lot: keep only three helpers—get_endpoint(...
now let desing the transform layer - i want the relevant data fields to b...
That's a clean analytical grain: one row per finished match. I'd have...
now lets design the load.py module - the output of this module should b...
For load.py, I'd keep it very small. Its job should be only: take the...



AI_Data_Engineer_Candidate_...

here is my assingment for a data eng. role , our first step is to break down the assingment:

1. define the areas where their should be more time-invesment and depth
2. must have vs nice to have
3. how to prioratize time there are 48 hours

6.9.2026, 19:11:54



Yes. The assignment makes the prioritization unusually clear because it gives explicit evaluation weights and says that a **smaller, well-designed, reliable solution is preferable to an overbuilt one.** `AI_Data_Engineer_Candidate_Assignment.docx`

The key is therefore **not to divide the 48 hours evenly across features.** We should allocate time according to what they're actually evaluating.

1. What are they really testing?

At a high level, this is not a "build a nice Streamlit dashboard" assignment.

It's an **end-to-end data engineering assignment with a product layer:**

API → Raw Data → ETL → Data Model → Quality → Analytics → Streamlit

The objective explicitly mentions extraction/transformation, analytical design, validation, and business-facing presentation.

`AI_Data_Engineer_Candidate_Assignment.docx`

Their scoring reinforces this:

Area	Weight	Recommended attention
End-to-End Ownership	25%	🔴 Very high
Architecture & ETL Design	25%	🔴 Very high
Analytics & Product Thinking	20%	🟡 High

Area	Weight	Recommended attention
Data Quality & Reliability	15%	🟡 High
AI Interaction	15%	🟡 High

So **50% of the score is essentially "can this person design and deliver a solid data pipeline?"** `AI_Data_Engineer_Candidate_Assignment.docx`

2. Where I would invest the most depth

🔴 Priority 1 - ETL architecture and data model

This deserves the deepest engineering work.

They explicitly expect extraction, cleaning, normalization, joins/aggregation/modeling, local persistence, and reproducibility. `AI_Data_Engineer_Candidate_Assignment.docx`

I'd want the reviewer to see that you thought about things like:

Extraction

- API interaction
- error handling
- raw persistence
- reproducibility

Transformation

- normalization
- types/schema
- missing data
- business rules
- derived columns

Load/model

- clean analytical dataset
- deterministic outputs
- sensible grain
- clear separation of raw vs processed

And, where relevant to the API, demonstrate engineering judgment around **incremental extraction, reruns/idempotency, API failures, changing source**

data, and avoiding unnecessary requests.

You don't need enterprise infrastructure. You need to demonstrate that you **think like a data engineer**.

3. Priority 2 - Complete end-to-end execution

This is extremely important because **End-to-End Ownership is 25%**.

Their required execution flow is deliberately simple:

```
git clone
  ↓
pip install -r requirements.txt
  ↓
python etl.py
  ↓
streamlit run app.py
```

AI_Data_Engineer_Candidate_Assignment.docx

A reviewer should be able to clone the repo and run those commands without touching your code.

That means I'd prioritize:

- clean repository
- correct requirements.txt
- no hard-coded local paths
- no manual preprocessing
- deterministic ETL
- sensible failure messages
- raw/processed directories
- working Streamlit app
- README matching reality

over sophisticated features.

A beautiful dashboard attached to a fragile pipeline would be a poor trade.

4. Priority 3 - Data quality and reliability

This is only 15% explicitly, but it also strengthens the ETL/architecture score.

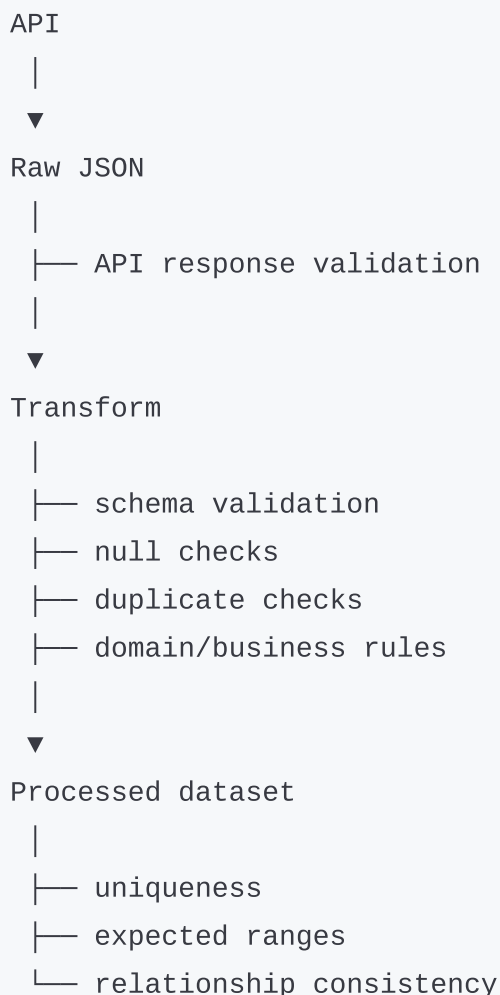
The assignment specifically calls out:

missing values, duplicates, invalid ranges, unexpected categories, API failures, or broken relationships. `AI_Data_Engineer_Candidate_Assignment.docx`

So don't implement generic checks just to say "I have data validation."

The checks should relate to your data.

For example:



3-6 meaningful checks > 20 superficial checks.

5. Priority 4 - Business question and analytics

This is worth **20%**, so the API shouldn't just be selected because it's easy to access.

They intentionally leave the business use case, API, data model, and dashboard direction to you because those decisions are themselves part of the evaluation.

AI_Data_Engineer_Candidate_Assignment.docx

The product should have a simple sentence like:

*"This application helps **X** understand **Y** so they can make **Z decision**."*

Then your data model, transformations, KPIs, and dashboard all support that question.

That's much stronger than:

"Here is some interesting API data."

6. Priority 5 - Streamlit

Streamlit matters, but I'd deliberately **cap the amount of time spent on UI polish**.

They want a business-facing application with clear data, relevant filters, and understandable insights. AI_Data_Engineer_Candidate_Assignment.docx

Must have:

Headline KPIs



Key business trend



Breakdown / comparison



Interactive filters



Short written insight

Nice to have:

- fancy CSS
- custom themes
- animations
- complicated navigation
- many pages
- excessive visualizations

A reviewer should understand the main insight within about **30 seconds**.

7. Must-have vs nice-to-have

I'd define the scope boundary now.

Component	Must Have	Nice to Have
API	Reliable public API	Multiple APIs
Extraction	Error handling + raw persistence	retries/backoff/caching sophistication
ETL	Clean modular pipeline	elaborate framework
Raw layer	✓	-
Processed layer	✓	-
Data model	Clear analytical grain	star schema if genuinely useful
Incremental logic	If appropriate to source	sophisticated state management
Data validation	Several meaningful checks	validation framework
Tests	Core transform/business logic	exhaustive coverage
Analytics	Clear business KPIs	advanced ML

Component	Must Have	Nice to Have
Streamlit	Clean functional dashboard	custom styling
Documentation	Excellent README	extra docs site
Architecture diagram	Simple, clear	elaborate diagrams
Logging	Basic useful logging	observability stack
Docker	✗	Explicitly unnecessary
Cloud DB	✗	Explicitly unnecessary
CI/CD	✗	Nice bonus only
AI transcript	Mandatory	-

Note that `/tests/` itself is marked optional in their repository structure, while data-quality logic is explicitly required. `AI_Data_Engineer_Candidate_Assignment.docx`

So I'd absolutely write some focused tests, but I would **not sacrifice pipeline completeness to chase high test coverage**.

8. 48-hour prioritization

I would **not interpret "48 hours" as 48 hours of coding**.

I'd target roughly **20-26 focused working hours**, leaving buffer for debugging and review.

A good allocation:

Phase	Time
Problem/API exploration	2 h
Architecture + data model design	2 h
Extraction	3 h
Transformation/modeling	4 h
Data quality + reliability	2.5 h

Phase	Time
Tests	1.5 h
Analytics/business metrics	2 h
Streamlit	3 h
README + architecture documentation	2 h
End-to-end testing / cleanup	2 h
Buffer	2 h
Total	26 h

The remaining elapsed time gives you room to sleep, rethink decisions, and recover from unexpected API/debugging problems.

9. The most important milestone structure

Rather than thinking in terms of "Day 1 = ETL, Day 2 = dashboard", I'd use gates.

Gate 1 - ~4 hours

You should have:

business question + API + data grain + architecture

Do **not** spend half a day choosing an API.

Gate 2 - ~10 hours

You should have:

```
API
↓
raw/
↓
transform
↓
processed/
```

working end-to-end.

No Streamlit yet if the pipeline isn't solid.

Gate 3 - ~16 hours

You should have:

```
ETL ✓  
Data model ✓  
Data quality ✓  
Basic tests ✓  
Analytics outputs ✓
```

At this point the engineering core is essentially submission-worthy.

Gate 4 - ~21 hours

Add:

```
Streamlit  
├─ KPIs  
├─ charts  
├─ filters  
└─ insights
```

Gate 5 - ~26 hours

Stop adding features.

Only:

clone → install → ETL → Streamlit → inspect → fix → README → final Git cleanup

This last phase is where many candidates probably lose points.

10. One unusual thing we should take very seriously: the AI transcript

They are explicitly evaluating **how you use AI**, not merely whether you used it.

They want evidence of how you broke down the assignment, selected the API, designed ETL/data modeling, debugged and validated the implementation, and importantly **challenged/refined AI suggestions rather than blindly accepting them**. `AI_Data_Engineer_Candidate_Assignment.docx`

That means this conversation itself is effectively **part of your submission**.

So our workflow should intentionally show reasoning such as:

```
Requirement
  ↓
Possible approaches
  ↓
Trade-offs
  ↓
Decision
  ↓
Implementation
  ↓
Validation
  ↓
Critique / improvement
```

rather than:

```
"Write extract.py"

"Write transform.py"

"Make Streamlit dashboard"
```

The former demonstrates engineering judgment. The latter mostly demonstrates that you know how to ask an LLM for code.

My priority hierarchy

If time starts running out, protect things in approximately this order:

- 1. Working end-to-end product**
- 2. ETL architecture + data model**
- 3. Reliability/data quality**

4. Strong business question + meaningful analytics
5. README/reproducibility
6. Focused tests
7. Streamlit polish
8. Everything else

And the assignment itself gives us permission to make that trade-off: they explicitly say they value **clarity, ownership, and practical judgment over unnecessary complexity**. `AI_Data_Engineer_Candidate_Assignment.docx`

The next step should therefore **not be coding yet**. We should define a compact **evaluation-driven architecture and API/use-case selection rubric** so that every feature we build can be traced to a scoring criterion.

6.9.2026, 19:11:56



lets find API that has fewest barriers im thinking sports data

6.9.2026, 19:13:57



Sports is a good direction here because it can give you a clear business question, repeated entities like teams/matches/seasons, and enough structure to show ETL quality without needing a huge system. I'll optimize for **no key, no payment, simple JSON, stable endpoints, and enough data depth for meaningful modeling.**

Thinking



Searching the web

football-data.org docs.balldontlie.io fifa.balldontlie.io thesportsdb.com
ncaab.balldontlie.io lab.balldontlie.io nfl.balldontlie.io balldontlie.io
ncaaf.balldontlie.io lol.balldontlie.io

- Searched 10 websites

✓ Worked for 8s

Yes - for this assignment, **sports data is a strong choice**, but I would optimize for **engineering simplicity first, richness second.**

The best fit I found is **OpenLigaDB**.

It is free, returns JSON, requires **no authentication at all for reading data**, and exposes simple football endpoints for matches, teams, standings, matchdays, goal scorers, and historical seasons. ([OpenLigaDB API](#))

Best options

API	Auth barrier	Free limitations	Data richness	Assignment fit
OpenLigaDB	None	Community-maintained data	Good	★★★★★

API	Auth barrier	Free limitations	Data richness	Assignment fit
TheSportsDB	Shared free key 123	30 req/min, some endpoints restricted	Very good	★★★★★
football-data.org	Account + API key	12 competitions, 10 req/min	Good	★★★☆☆
BALLDONTLIE	Account + API key	5 req/min; many useful stats paid	Good	★★☆☆☆

TheSportsDB is also quite easy: its free v1 API uses the public key 123 and permits 30 requests/minute. But its API naming is somewhat messy and more advanced methods are restricted to premium. ([TheSportsDB](#))

football-data.org is clean and established, but we'd need registration/API-key configuration, and the free plan is limited to 12 competitions and 10 calls/minute. ([Football Data](#))

I would reject BALLDONTLIE for this assignment because although it's free at entry level, it requires an account/key, allows only five requests/minute, and locks many analytically interesting endpoints behind paid tiers. ([Balldontlie](#))

Why OpenLigaDB fits the assignment particularly well

The biggest advantage isn't just "free."

It removes an entire category of failure:

```

No registration
No API key
No .env
No secrets
No reviewer configuration
No payment
No private credentials

python etl.py
↓
works

```

That aligns almost perfectly with the assignment requirement that the reviewer should be able to run the project without manual data preparation.

AI_Data_Engineer_Candidate_Assignment.docx

OpenLigaDB explicitly says reading the API requires **no authentication**.
([OpenLigaDB](#))

And the API is structurally useful for ETL. For example:

```
/getavailableleagues/{season}

/getmatchdata/{league}/{season}

/getmatchdata/{league}/{season}/{matchday}

/getavailableteams/{league}/{season}

/getavailablegroups/{league}/{season}

/gettbltable/{league}/{season}

/getgoalgetters/{league}/{season}

/getcurrentgroup/{league}
```

([OpenLigaDB API](#))

That gives us enough structure to demonstrate actual extraction design rather than just downloading one giant JSON blob.

Bundesliga is probably the safest dataset

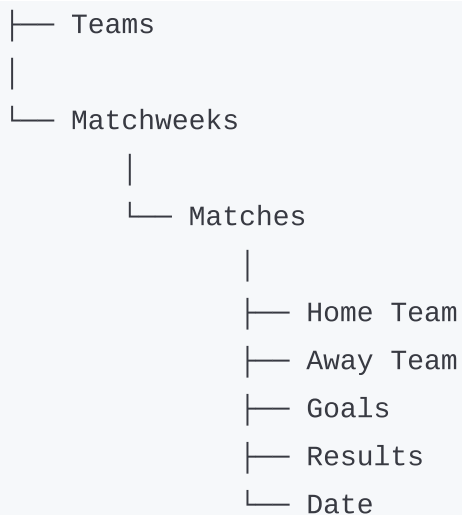
For example, the current 2026/27 Bundesliga is available as:

```
leagueShortcut = bl1
season = 2026
```

OpenLigaDB currently lists that league with a quality index of 100. ([OpenLigaDB](#))

So the underlying model becomes very intuitive:

```
League
|
```



That's excellent assignment data because it contains nested JSON that we can legitimately **flatten and model**.

More importantly, it gives us a natural ETL problem

Instead of:

"Download football data and show charts."

we could build something that behaves like a small production pipeline:

```
OpenLigaDB
  ↓
Extract
  ↓
data/raw/
matchweek_1.json
matchweek_2.json
...
  ↓
Transform
  ↓
matches
teams
team_performance
  ↓
Quality checks
  ↓
data/processed/
```


↓
Streamlit

And because matchweeks evolve over time, we get a legitimate reason to discuss:

incremental ingestion → completed vs unfinished matches → reruns → idempotency → API failures → historical immutability → data validation.

That is much more valuable for a Data Engineer assignment than choosing an API with 100 different player statistics.

One caution

OpenLigaDB is **community-maintained**, so its data can itself contain quality issues. ([OpenLigaDB](#))

But for this assignment I actually view that as an advantage.

Instead of pretending the upstream source is perfect, we can demonstrate:

```
API data
  ↓
Schema validation
  ↓
Duplicate detection
  ↓
Missing score detection
  ↓
Finished-match validation
  ↓
Invalid score/range checks
  ↓
Processed analytical data
```

That directly addresses the assignment's explicit requirement to identify and surface meaningful data-quality problems. `AI_Data_Engineer_Candidate_Assignment.docx`

My recommendation

I'd choose:

OpenLigaDB → Bundesliga → team/match performance analytics product

and stop searching for APIs unless we discover a concrete limitation.

The next decision is more important than the API itself: **what business-facing product are we building from football match data?** We should generate 4-5 possible business questions and select the one that gives us the strongest combination of **ETL complexity + analytics + Streamlit**, without increasing implementation risk.

Sources

[Swagger UI](#) api.openligadb.de

[TheSportsDB Free Sports API Documentation - TheSportsDB.com](#) thesportsdb.com

[football-data.org - Pricing](#) football-data.org

[BALLDONTLIE NBA API](#) docs.balldontlie.io

[OpenLigaDB](#) openligadb.de

6.9.2026, 19:13:59



ive looked throught the website and i prefer to analyze the premier leauge ongoing 26-27 season - dymanic data can add a nice layer of the complexity to the pipeline (may impress the reviewws)

6.9.2026, 19:16:17



Thinking



Searching the web

openligadb.de

api.openligadb.de

beta.openligadb.de

openligadb.de

- **Searched 4 websites**

✓ Worked for 7s

I agree with that choice. The **ongoing Premier League 2026/27 season** is actually better for this assignment than a completed historical season because the source changes over time, which gives us a legitimate reason to design the ETL around **mutable vs stable data** rather than just downloading a static dataset.

OpenLigaDB currently lists **Premier League 2026/2027** with `leagueShortcut = p1`, `season = 2026`, and a quality index of 100. ([OpenLigaDB](#)) The API also exposes both season-level and matchweek-level match endpoints, which is exactly what we need for incremental extraction. ([OpenLigaDB API](#))

The interesting engineering problem becomes:

OpenLigaDB



What has changed since the last run?



Fetch only relevant matchweeks



Persist raw API responses



Transform completed matches



```
↓
Validate
↓
Update analytical dataset
↓
Streamlit
```

That is much stronger than simply:

```
GET whole season
↓
CSV
↓
dashboard
```

The assignment explicitly evaluates architecture, reproducibility, data quality, and operational thinking, so dynamic data lets us demonstrate those dimensions naturally.

`AI_Data_Engineer_Candidate_Assignment.docx`

The main design idea I would build around is **matchweek state**. Because today's source says the Premier League is currently at matchweek 3, some historical rounds can eventually become immutable while the current round remains mutable.

([OpenLigaDB](#))

Conceptually:

```
Run ETL
↓
GET current matchweek C
↓
For matchweek 1 ... C
↓
Has this matchweek already been sealed complete?
|
├─ YES → skip API request
|
└─ NO
    ↓
    fetch matchweek
    ↓
    save raw JSON
```

```
↓
are all matches finished?
|
├─ YES → mark complete
└─ NO  → keep open for next run
```

This gives us several **real data-engineering concepts** without artificially overengineering the solution:

Incremental extraction - don't repeatedly pull immutable history.

Late-arriving/changeable data - today's or current matchweek's fixtures/results can change, so we refresh them.

Idempotency - running `python etl.py` repeatedly should produce the same analytical state given the same API state.

Raw persistence - preserve source payloads for reproducibility/debugging.

State tracking - a tiny local JSON ledger can record which matchweeks are considered complete.

Failure recovery - if an API request fails halfway through, don't corrupt previously valid data.

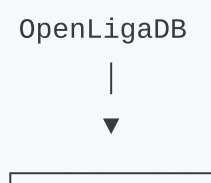
That is exactly the kind of complexity I would want in this assignment: **complex enough to show judgment, but small enough that a reviewer can understand it immediately.**

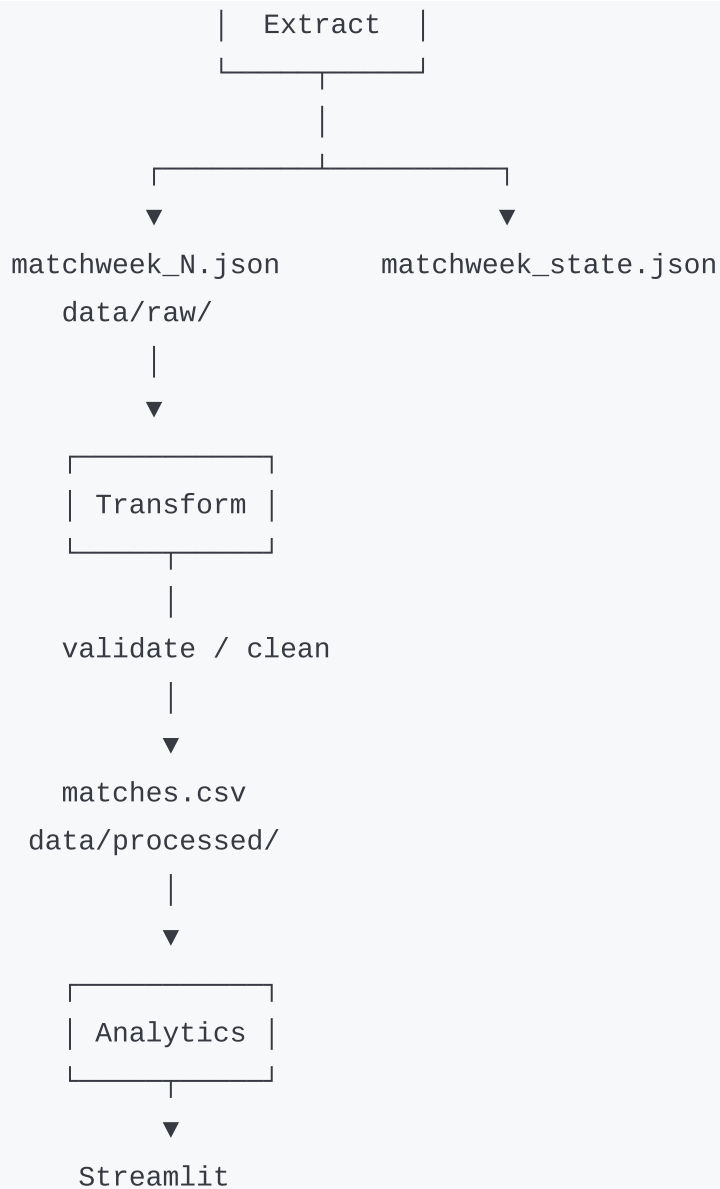
There is one important nuance I'd keep in mind: we should not make "dynamic pipeline" synonymous with "complicated pipeline." Their final note explicitly says a smaller reliable design is preferable to unnecessary complexity.

`AI_Data_Engineer_Candidate_Assignment.docx`

So I would avoid things like Airflow, databases, scheduling infrastructure, Kafka, Docker, or elaborate CDC. A local state file and carefully designed extraction logic are enough to demonstrate the concept.

The architecture could ultimately look something like this:





I would therefore **lock in the source now**:

OpenLigaDB → pl → 2026/27 ongoing Premier League season.

The next decision should be the **actual data product/business question**. That's important because it determines what our processed data model needs to contain. We should now inspect exactly what OpenLigaDB gives us per Premier League match, then derive 3-5 business-facing product ideas from the fields that actually exist rather than choosing an analysis first and forcing the API to fit it.

Sources

[Ligen-Übersicht](https://openligadb.de) openligadb.de



match_data_pl_2.json

here is a partial json response from this api for a specific matchweek (MW2)

6.9.2026, 19:24:20



This partial response is actually enough to confirm that the dataset is viable, but it also reveals an important constraint: **the match payload itself is relatively lean.**

From this MW2 sample, each fixture gives us:

- fixture_id
- kickoff date
- status
- home/away team names
- home/away final scores

and the response-level metadata includes league, country, matchday, and season.

match_data_pl_2.json The individual fixture records are similarly compact.

match_data_pl_2.json

That means the strongest product direction is probably **team and match performance analytics**, not advanced player analytics or xG-style analysis.

A clean processed match table could look like this:

```
fixture_id
season
matchday
date
status
home_team
away_team
home_score
away_score
```



```
result
winner
total_goals
goal_difference
```

For example, from the Tottenham-Everton fixture we can derive:

```
result = HOME_WIN
winner = Tottenham Hotspur
total_goals = 2
goal_difference = 2
```

The same logic applies consistently to the other fixtures. `match_data_p1_2.json`

What makes this useful for the assignment is that we can then create a second analytical layer at **team x matchweek grain**:

```
season
matchday
team
opponent
venue
goals_for
goals_against
goal_difference
points
win
draw
loss
```

Then aggregate that into:

```
team
matches_played
wins
draws
losses
goals_for
goals_against
goal_difference
points
```

```
points_per_game
avg_goals_scored
avg_goals_conceded
```

That gives us much more interesting Streamlit possibilities even though the raw API is simple.

The dynamic aspect is still valuable. Because fixtures have a `status` field such as `FT`, the ETL can distinguish completed from unfinished matches. `match_data_p1_2.json`

That supports a sensible rule like:

```
if status == "FT":
    include in analytical aggregates
else:
    persist raw fixture
    exclude from finalized performance metrics
```

This also gives us meaningful validation rules:

```
fixture_id must be unique

matchday must be positive

home_team != away_team

if status == FT:
    home_score must not be null
    away_score must not be null
    scores must be >= 0

fixture date must parse successfully
```

So the dataset is simple, but that isn't necessarily bad. The assignment explicitly values a reliable, well-designed solution over unnecessary complexity.

`AI_Data_Engineer_Candidate_Assignment.docx`

The one thing I would check before locking the analytical product is whether the **actual OpenLigaDB response contains additional fields that were removed from this partial sample** - things like halftime results, goals, team IDs, locations, or

nested result objects. If those exist, they could materially expand the analytics without increasing API complexity.

For now, though, I'd say this payload supports a very solid core product:

“Premier League Season Pulse - track evolving team performance across the ongoing 2026/27 season.”

That product could focus on **form, points progression, goal production, defensive performance, home/away splits, and matchweek trends.**

6.9.2026, 19:24:21



lets desing a simple exctarction module (extract.py) that pulls the data from the api :

1. pulled data is up to the current MW
2. keep the process incremental (no runovers or unnesecary data fetching)
3. keep only the relevant data from the json for the analysis

6.9.2026, 19:28:59



I'd keep `extract.py` responsible for only three things: **discover current matchweek, fetch only matchweeks that still need refreshing, and persist a compact per-matchweek JSON cache**. One design choice matters here: we should preserve enough source data for reproducibility, but not mix analytical derivations into extraction.

Thinking



Searching the web

`api.openligadb.de`

- **Designing the module**

I'm outlining a simple state-aware workflow before writing code. I'm balancing minimal fetching with reliability: sealed matchweeks can be skipped, while the current week and any incomplete postponed weeks remain eligible for refreshes.

- **Searched 1 website**

✓ Worked for 15s

Yes. I'd make the extraction logic deliberately simple, but I would change one part of point 3:

`extract.py` **should not strip the API response down to analytical fields**. The assignment explicitly asks you to persist raw API data for reproducibility.

`AI_Data_Engineer_Candidate_Assignment.docx` Field selection belongs in `transform.py`.

That gives us a cleaner separation:

```
extract.py
API → raw JSON

transform.py
raw JSON → relevant fields → processed data
```

For the incremental extraction itself, this is the design I recommend.

Extraction algorithm

OpenLigaDB gives us exactly the endpoints we need:

```
GET /getcurrentgroup/pl
```

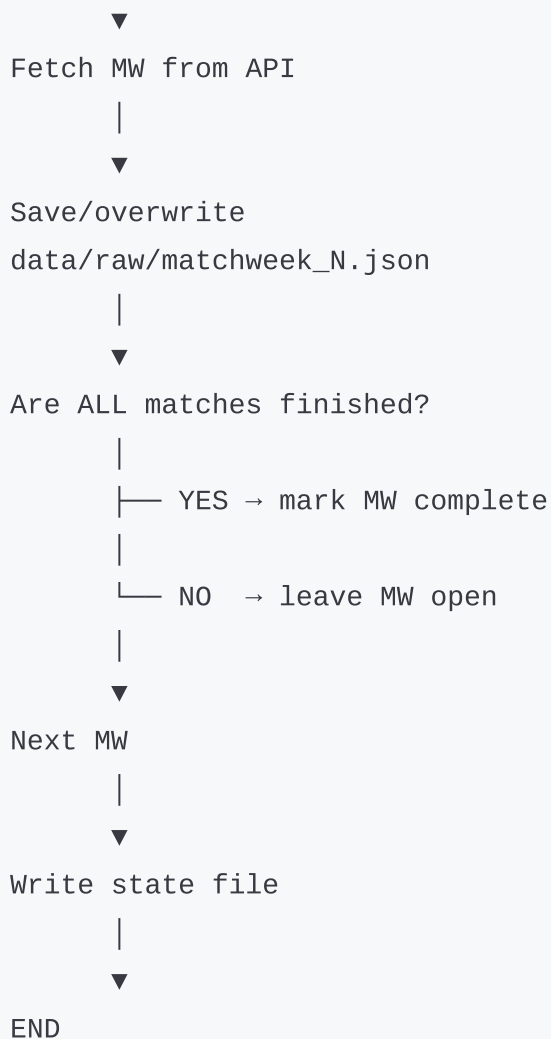
to determine the current matchweek, and:

```
GET /getmatchdata/pl/2026/{matchweek}
```

to retrieve one matchweek. ([OpenLigaDB API](#))

Then:

```
START extract
  |
  ▼
Get current MW → C
  |
  ▼
Read local state file
  |
  ▼
For MW = 1 ... C
  |
  ▼
Is MW already sealed complete?
  |
  YES → SKIP
  |
  NO
```



The resulting raw directory might be:

```
data/  
├─ raw/  
    ├── matchweek_1.json  
    ├── matchweek_2.json  
    ├── matchweek_3.json  
    └─ matchweek_state.json
```

And the state file stays tiny:

```
{  
  "1": {  
    "complete": true  
  },  
  "2": {  
    "complete": true  
  },  
}
```

```
"3": {  
  "complete": false  
}
```

Suppose we're currently in MW3.

On the **first run**:

```
GET current MW    → 3  
  
GET MW1           → save → complete  
GET MW2           → save → complete  
GET MW3           → save → incomplete
```

On the **second run**:

```
GET current MW    → 3  
  
MW1 → complete → SKIP  
MW2 → complete → SKIP  
MW3 → incomplete → GET again
```

So instead of downloading the whole season every time, the second ETL execution performs essentially:

```
1 lightweight current-MW request  
+  
1 current/open matchweek request
```

That's the incremental behavior we want.

Important edge case: postponed matches

We should **not simply assume that** `MW < current_MW` **means complete**.

Suppose MW2 had:

```
9 finished games  
1 postponed game
```

Then:

```
{
  "2": {
    "complete": false
  }
}
```

Even when the league reaches MW3, MW2 continues being refreshed:

```
MW1 → complete → skip
MW2 → incomplete → fetch
MW3 → current → fetch
```

Once the postponed match eventually finishes:

```
MW2 → all matches finished → complete = true
```

and future executions stop fetching it.

That's a small implementation detail that demonstrates good operational thinking.

What counts as "complete"?

Your sample already gives us exactly the information needed:

```
{
  "fixture_id": 1021,
  "date": "2026-08-22T12:30:00Z",
  "status": "FT",
  ...
}
```

match_data_pl_2.json

Conceptually:

```
complete = all(
    fixture["status"] == "FT"
    for fixture in fixtures
)
```


For the real OpenLigaDB payload, we'll map this to its actual finished-match field rather than assuming your simplified field names.

What `extract.py` should contain

I'd keep it to roughly five responsibilities:

```
get_current_matchweek()
    |
    ▼
load_state()
    |
    ▼
fetch_matchweek()
    |
    ▼
is_matchweek_complete()
    |
    ▼
save_raw() + save_state()
```

Something structurally like:

```
BASE_URL = "https://api.openligadb.de"
LEAGUE = "pl"
SEASON = 2026
```

```
def get_current_matchweek():
    ...
```

```
def fetch_matchweek(matchweek):
    ...
```

```
def load_state():
    ...
```

```
def save_raw(matchweek, data):
```

```

...

def is_complete(data):
    ...

def extract():
    current_mw = get_current_matchweek()
    state = load_state()

    for mw in range(1, current_mw + 1):

        if state.get(str(mw), {}).get("complete"):
            continue

        data = fetch_matchweek(mw)

        save_raw(mw, data)

        state[str(mw)] = {
            "complete": is_complete(data)
        }

    save_state(state)

```

That's essentially the entire algorithm.

One refinement I would add

I'd **always refresh the current/live matchweek**, even in the unlikely case the state file says it is complete.

So the condition becomes conceptually:

```

if is_sealed_complete and mw != current_mw:
    continue

```

Why?

Because the live round is the most likely place for upstream corrections. We're spending one API call for a little additional safety.

Therefore:

```
Historical sealed MW → frozen
Incomplete historical MW → refresh
Current MW → always refresh
New MW → fetch
```

That's a very understandable policy to explain to a reviewer.

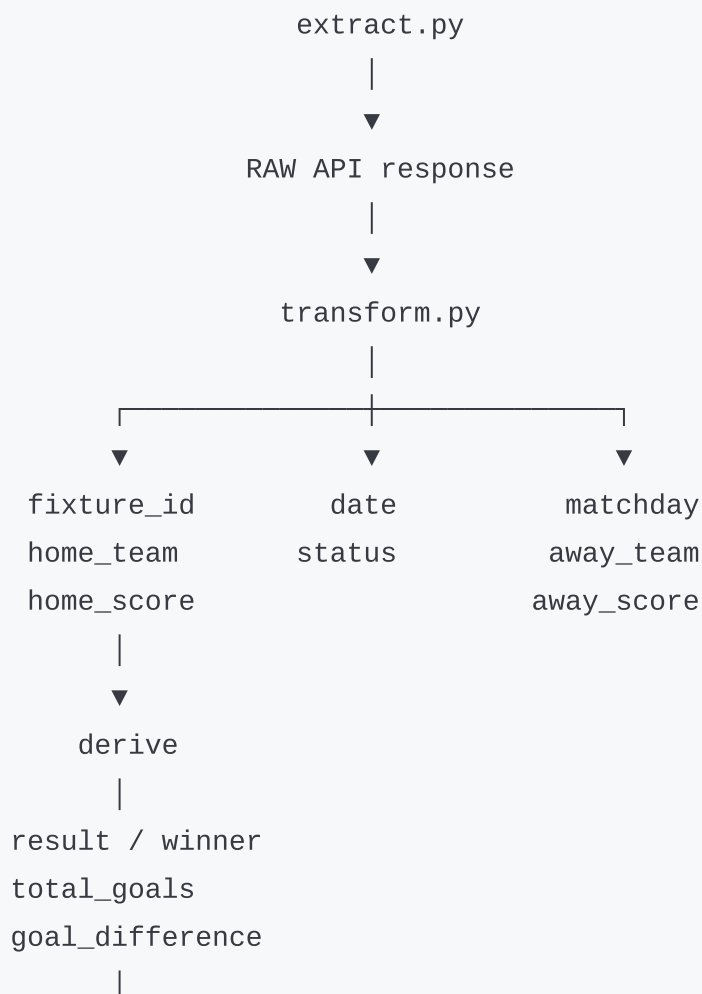
Where point 3 belongs

Your original requirement was:

keep only the relevant data from the JSON for the analysis

I would move that explicitly into `transform.py`.

So:





data/processed/matches.csv

That separation is stronger because if we later realize we need another API field, **we don't have to re-download historical data**. We just change `transform.py` and rerun against the raw cache.

That's exactly why retaining raw data is valuable.

So I would lock in these three extraction principles:

Incremental: frozen history is not downloaded repeatedly.

Mutable-aware: incomplete and current matchweeks continue refreshing.

Reproducible: extraction preserves raw source data; analytical field selection happens downstream.

And importantly, OpenLigaDB also exposes a

`/getlastchangedate/{league}/{season}/{matchweek}` endpoint. ([OpenLigaDB API](#)) We *could* use it to make change detection even smarter, but I would classify that as **nice-to-have**. For a 48-hour assignment, the state-based algorithm above is easier to understand, easier to test, and already demonstrates the engineering principle very well.

Sources

[Swagger UI](#) `api.openligadb.de`

6.9.2026, 19:29:00



for a given matchweek - there may be games yet to be played (different time) so i want to extract only the matches played with given data

6.9.2026, 19:31:02



Exactly. In that case, I'd separate **matchweek completeness** from **which matches are usable for analysis**.

Your MW2 sample already supports this because each fixture has a `status`, with completed games marked `FT`. `match_data_pl_2.json`

So for a current matchweek like:

```
MW3
├─ Match 1 → FT
├─ Match 2 → FT
├─ Match 3 → FT
├─ Match 4 → upcoming
├─ Match 5 → upcoming
└─ ...
```

the extraction run should fetch MW3, but only the **finished matches** should continue into the analytical dataset.

The important distinction is:

```
API response for MW3
|
▼
Persist raw MW3 response
|
▼
Transform
├─ FT      → keep
└─ not FT  → ignore for analytics
```

I would still keep the full matchweek response in `data/raw/matchweek_3.json`, including future fixtures. That preserves the original source response and complies better with the assignment's reproducibility requirement.

`AI_Data_Engineer_Candidate_Assignment.docx`

Then `transform.py` can do something as simple as:

```
played_matches = [  
    match  
    for match in fixtures  
    if match["status"] == "FT"  
]
```

and only those matches become rows in `matches.csv`.

This also makes the incremental logic cleaner. Suppose MW3 unfolds across three days:

```
Saturday  
MW3 → 6/10 finished  
    ↓  
processed dataset gets those 6  
  
Sunday  
MW3 refreshed → 9/10 finished  
    ↓  
processed dataset now gets 9  
  
Monday  
MW3 refreshed → 10/10 finished  
    ↓  
processed dataset gets all 10  
    ↓  
MW3 becomes sealed
```

After that:

```
MW3 complete → future ETL runs skip it
```

So the state isn't really tracking "do we have this matchweek?" It's tracking:

Can this matchweek still produce new analytical data?

That's a better way to think about it.

I would therefore define the extraction policy as:

```
For MW 1 → current MW:

    if historical MW is sealed complete:
        skip

    otherwise:
        fetch the matchweek
        save raw response

        if every fixture is FT:
            mark MW complete
```

Then transformation independently says:

```
for each raw match:
    if status != FT:
        skip

    else:
        transform and include
```

This is also naturally **idempotent**. On Saturday you might write six MW3 matches to the processed output. On Sunday you don't append three more blindly; `transform.py` rebuilds the analytical dataset from the current raw cache:

```
matchweek_1.json └─
matchweek_2.json ┤→ transform → matches.csv
matchweek_3.json └─
```

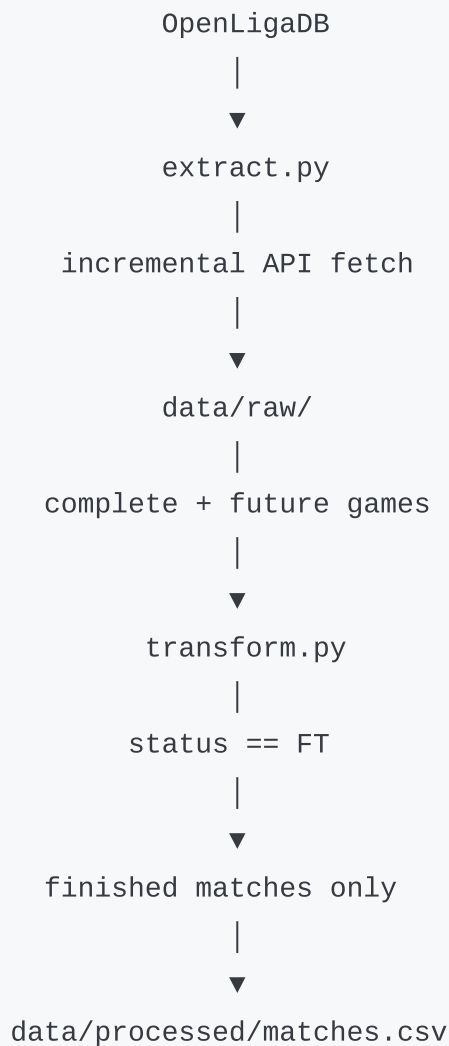
So you never risk:

```
Saturday MW3 games
+
Sunday same MW3 games
```

=
duplicates

Instead the newest `matchweek_3.json` overwrites the previous raw snapshot, and transformation produces the current truth.

That's a very clean architecture for the assignment:



One terminology adjustment I'd make to your requirement is therefore:

Extract matchweeks up to the current matchweek, persist the raw response, and expose only completed matches to the analytical layer.

That preserves a clean responsibility boundary between **extraction** and **transformation**, which will make the design easier to justify to the reviewers.

6.9.2026, 19:31:05



generate the extract module

6.9.2026, 19:31:54



I'll generate it with the responsibility split we agreed on: `extract.py` will fetch incrementally and persist raw matchweek payloads; it will use match completion only to decide whether a matchweek can be frozen. Filtering finished matches for analytics stays in `transform.py`.

I've verified the live API shape we need: the current-round endpoint is `/getcurrentgroup/p1`, and matchweek extraction is `/getmatchdata/p1/2026/{mw}`. I'm using OpenLigaDB's native `matchIsFinished` flag to decide whether a round can be sealed, while preserving the full raw response for reproducibility.

Thinking



Searching the web

api.openligadb.de

beta.openligadb.de

openligadb.de

- Searched 3 websites

✓ Worked for 14s

Generated the extraction module: [download](#) `extract.py`.

The module implements the policy we designed: it discovers the current Premier League matchweek, skips sealed historical rounds, refreshes incomplete/current rounds, writes raw matchweek JSON atomically, and tracks completion in `matchweek_state.json`. It deliberately preserves the raw API response rather than filtering analytical fields during extraction, which matches the assignment's reproducibility requirement. `AI_Data_Engineer_Candidate_Assignment.docx`

One important detail: the analytical filtering for **played matches only** should happen next in `transform.py`, using OpenLigaDB's finished-match indicator. That keeps the architecture clean: `extract.py` = source acquisition, `transform.py` = selection + cleaning + derived features.

6.9.2026, 19:31:56



this module is heavy and abit over complicate - use helper function to get the endpoint , write the state json and read the state

also i want the extract to return a list of all matches - we will use a caller function to plug the data into the transform component

6.9.2026, 19:38:32



Agreed. We can simplify it a lot: keep only three helpers—`get_endpoint()`, `read_state()`, `write_state()`—and let `extract()` own the loop. It will also return one flat list of all fetched/cached matches so `etl.py` can pass that directly into `transform()`.

This version is much leaner and keeps the flow easy to follow.

`extract()` now returns `all_matches`, so your caller can simply do:

```
all_matches = extract()
processed = transform(all_matches)
```

The state is also simplified to:

```
{
  "1": true,
  "2": true,
  "3": false
}
```

rather than nested objects.

Historical completed matchweeks are loaded from the local raw cache instead of hitting the API again; incomplete/current matchweeks are refreshed. The actual filtering of unfinished games should still happen inside `transform()`.

[Download the simplified `extract.py`](#)

6.9.2026, 19:38:34



now let desing the transform layer - i want the relevant data fields to be :

- match date - format DD-MM-YYYY
- kickoff time - formt HH-MM
- matchweek (MW4,MW5 ect...)
- home team name
- away team name
- home goals
- away goals
- result
- total goals
- goal difference

6.9.2026, 20:01:44



That's a clean analytical grain: **one row per finished match.**

I'd have `transform(all_matches)` do four things only: filter unfinished matches, flatten the nested source fields, format date/time, and derive the analytical columns.

The processed schema would be:

Field	Example
match_date	22-08-2026
kickoff_time	15-00
matchweek	MW2
home_team	Manchester City
away_team	Ipswich Town
home_goals	4
away_goals	1
result	HOME_WIN
total_goals	5

Field	Example
goal_difference	3

For `result`, I recommend using categorical values:

```
HOME_WIN
AWAY_WIN
DRAW
```

rather than storing something like `"4-1"`, because the actual score is already represented by `home_goals` and `away_goals`.

The core transformation logic should look conceptually like this:

```
def transform(all_matches):

    transformed = []

    for match in all_matches:

        if not match["matchIsFinished"]:
            continue

        home_goals = ...
        away_goals = ...

        if home_goals > away_goals:
            result = "HOME_WIN"
        elif home_goals < away_goals:
            result = "AWAY_WIN"
        else:
            result = "DRAW"

        transformed.append({
            "match_date": ...,
            "kickoff_time": ...,
            "matchweek": ...,
            "home_team": ...,
            "away_team": ...,
            "home_goals": home_goals,
```

```

        "away_goals": away_goals,
        "result": result,
        "total_goals": home_goals + away_goals,
        "goal_difference": abs(home_goals - away_goals),
    })

    return transformed

```

One detail I'd define explicitly now: `goal_difference` should probably be the **absolute match margin**, so a 4-1 game gives `3`, not `+3` for the home team or `-3` for the away team. Since this table is match-level rather than team-level, that interpretation is cleaner.

And the responsibility split stays very clear:

```

extract.py
API → list of raw matches

↓

transform.py
finished matches only
flatten fields
format fields
derive metrics

↓

list of processed match dictionaries

```

Then `load.py` can simply turn that list into a DataFrame and write `matches.csv`.

The next step is to map these fields precisely to the **actual OpenLigaDB JSON structure**, especially where final scores are stored, because the raw API uses nested result objects rather than necessarily exposing `home_score` / `away_score` directly.

6.9.2026, 20:01:47



now lets design the load.py module - the output of this module should be a clear csv which is placed inside the processed folder inside the data folder

6.9.2026, 20:03:30



For `load.py`, I'd keep it very small. Its job should be only:

take the transformed list → create a clean DataFrame → save

`data/processed/matches.csv`

That keeps the ETL responsibilities clean:

```
extract.py → raw match list
transform.py → cleaned/derived match list
load.py → processed CSV
```

A simple design:

```
from pathlib import Path

import pandas as pd

PROCESSED_DIR = Path("data/processed")
OUTPUT_FILE = PROCESSED_DIR / "matches.csv"

def load(matches):
    """
    Save transformed match data as a processed CSV.

    Parameters
    -----
    matches : list[dict]
        Output returned by transform().

    Returns
    -----
```

```

Path
    Path to the generated CSV file.
"""
PROCESSED_DIR.mkdir(parents=True, exist_ok=True)

df = pd.DataFrame(matches)

df.to_csv(
    OUTPUT_FILE,
    index=False,
    encoding="utf-8"
)

return OUTPUT_FILE

if __name__ == "__main__":
    print("Run load() through etl.py")

```

The resulting file would be:

```

data/
├── raw/
│   ├── matchweek_1.json
│   ├── matchweek_2.json
│   └── matchweek_state.json
└── processed/
    └── matches.csv

```

And the CSV would look like:

```

match_date,kickoff_time,matchweek,home_team,away_team,home_goals,away_goals,result
22-08-2026,15-00,MW2,Manchester City,Ipswich Town,4,1,HOME_WIN,5,3
24-08-2026,15-30,MW2,Aston Villa,Arsenal,0,2,AWAY_WIN,2,2

```

I'd also keep `load.py` free of business logic. No filtering, no goal calculations, no validation of match status there. All of that belongs upstream in `transform.py`.

Then your `etl.py` becomes very clean:

```
from src.extract import extract
from src.transform import transform
from src.load import load

def run_etl():
    raw_matches = extract()
    processed_matches = transform(raw_matches)
    output_file = load(processed_matches)

    print(f"ETL completed successfully: {output_file}")

if __name__ == "__main__":
    run_etl()
```

That separation is probably exactly what you want for this assignment: simple, readable, and easy for a reviewer to trace.

6.9.2026, 20:03:32